



Universidad de Buenos Aires

Facultad de Ingeniería

## Trabajo Práctico N.º 1 - File Transfer

[TA048] Redes

Cátedra Hamelin-Alvarez Curso martes (Lopez/Horn)

Segundo cuatrimestre del 2026

---

### Grupo 7

#### Integrantes

| Alumno                             | Padrón | Email              |
|------------------------------------|--------|--------------------|
| Javier Zardain                     | 102521 | jzardain@fi.uba.ar |
| Iñaki Vydra                        | 111505 | ivydra@fi.uba.ar   |
| Fiorella Chiara de Deus Nascimento | 112772 | fdedeus@fi.uba.ar  |
| Daniela Pareja                     | 100648 | dpareja@fi.uba.ar  |
| Nicolás Feldman                    | 113735 | nfeldman@fi.uba.ar |

# Índice

1. Introducción
2. Hipótesis y suposiciones realizadas
3. Implementación
4. Pruebas
5. Preguntas a responder
6. Dificultades encontradas
7. Conclusión

## 1. Introducción

El presente trabajo aborda el diseño y la implementación de una aplicación de transferencia de archivos basada en una arquitectura cliente-servidor, que permite realizar operaciones de subida y descarga. Su objetivo es explorar de forma práctica los principios de la transferencia confiable de datos y comprender los mecanismos necesarios para ofrecer este servicio sobre UDP.

Para ello, se define un protocolo de aplicación que establece los mensajes y las reglas de intercambio entre el cliente y el servidor, junto con dos estrategias de recuperación de errores: Stop & Wait y un mecanismo basado en confirmaciones selectivas (SACK). Mientras que Stop & Wait envía un segmento y espera su confirmación antes de continuar, la estrategia con SACK permite mantener varios segmentos en vuelo e identificar aquellos que deben retransmitirse.

Su implementación permite analizar las decisiones de diseño involucradas en una comunicación confiable y comparar el desempeño de ambas estrategias bajo distintas condiciones de red.

En este informe se detallan las hipótesis y suposiciones adoptadas, la arquitectura de la aplicación y los protocolos de comunicación implementados. Asimismo, se presentan las pruebas realizadas, el análisis de los resultados y las dificultades encontradas durante el desarrollo.

## 2. Hipótesis y suposiciones realizadas

### 2.1. Hipótesis y criterios de diseño

- **Identificación de sesiones.** Se considera suficiente identificar cada sesión mediante el par (*IP, puerto*) de origen observado por el servidor, sin incorporar un identificador adicional en los segmentos. Cada operación de subida o descarga crea su propio socket UDP y realiza una única transferencia. Por lo tanto, no se contemplan dos transferencias simultáneas desde el mismo socket.
- **Capacidad del servidor.** Se considera que un máximo de **50 sesiones activas**, con una cola de hasta **4096 datagramas por sesión**, es suficiente para las pruebas previstas en el trabajo práctico. Su suficiencia deberá contrastarse con las pruebas de concurrencia. Al alcanzar el límite de sesiones, el servidor descarta nuevas solicitudes de establecimiento; si se llena una cola, descarta el datagrama y su recuperación queda a cargo del protocolo confiable.

- **Modelo de concurrencia.** Se considera que un hilo dedicado por cada sesión de cliente es suficiente para el alcance del trabajo práctico y el límite de sesiones previsto. Por ello, no se incorpora un conjunto reutilizable de hilos trabajadores (thread pool). Esta decisión simplifica la atención de cada transferencia, a costa de mantener un hilo por sesión activa.
- **Exclusividad de los nombres de archivo.** El servidor no permite subir un archivo cuyo nombre ya exista en el directorio de almacenamiento o esté reservado por otra subida en curso. Esta decisión evita la sobrescritura y los conflictos entre clientes que intentan escribir sobre el mismo destino. Si se presenta alguna de estas condiciones, la solicitud se rechaza y se informa el motivo al cliente mediante el protocolo de aplicación.

## 2.2. Suposiciones sobre la red

- **Integridad de los datagramas recibidos.** Para el alcance del trabajo se supone que el checksum UDP está habilitado y que no se producen corrupciones que pasen inadvertidas a su verificación. Bajo este supuesto, el contenido de los datagramas recibidos coincide con el enviado, por lo que no se agrega un checksum propio al protocolo.
- **Ausencia de fragmentación IP en el entorno de prueba.** Se supone una red IPv4 con MTU de **1500 bytes en todos los enlaces del camino**, sin encapsulaciones adicionales que reduzcan la MTU efectiva. Se limita a **1400 bytes** el tamaño de los datos entregados a UDP, incluyendo las cabeceras de los protocolos propios. Al sumar los 8 bytes de cabecera UDP y los 20 bytes de una cabecera IPv4 sin opciones, el paquete IP ocupa como máximo **1428 bytes**, por debajo de la MTU asumida. Por ello, no se prevé fragmentación IP en estas condiciones.
- **Estabilidad de las direcciones.** Se supone que la IP y el puerto de origen observados por el servidor no cambian durante la transferencia. Un cambio de dirección o puerto no se interpreta como una continuación de la sesión existente.
- **Reutilización de puertos.** Se supone que un par (*IP, puerto*) no se reutiliza para una nueva sesión mientras puedan llegar datagramas de una transferencia anterior. El servidor conserva temporalmente la última respuesta de una sesión cerrada para atender retransmisiones rezagadas, pero el protocolo no incorpora un identificador que permita distinguir generaciones de sesiones con la misma dirección.

## 2.3. Simplificaciones de archivos y almacenamiento

- **Espacio de nombres plano.** Los archivos del servidor se identifican por su nombre dentro de un único directorio de almacenamiento. No se admiten rutas a subdirectorios, nombres con separadores, enlaces simbólicos ni nombres terminados en *.tmp*. Estas restricciones simplifican la resolución de destinos y reservan el sufijo de los archivos temporales.
- **Archivos de origen estables.** Se supone que ningún proceso externo modifica el archivo de origen mientras se transfiere. La implementación realiza una lectura secuencial, sin crear una copia inmutable ni bloquear modificaciones externas. Este supuesto evita que el contenido cambie entre el anuncio de su tamaño y la finalización del envío.
- **Transferencias sin reanudación.** No se implementa la continuación de una transferencia interrumpida desde un desplazamiento previo. Los datos recibidos se escriben en un archivo temporal y se publican con su nombre definitivo al confirmar la operación. Ante una cancelación gestionada por el programa, el temporal se elimina; si queda un temporal por una terminación abrupta, una nueva escritura sobre ese destino se rechaza hasta resolverlo.

Se supone que el sistema de archivos admite enlaces duros, utilizados para publicar el archivo completo sin sobrescribir un destino existente.

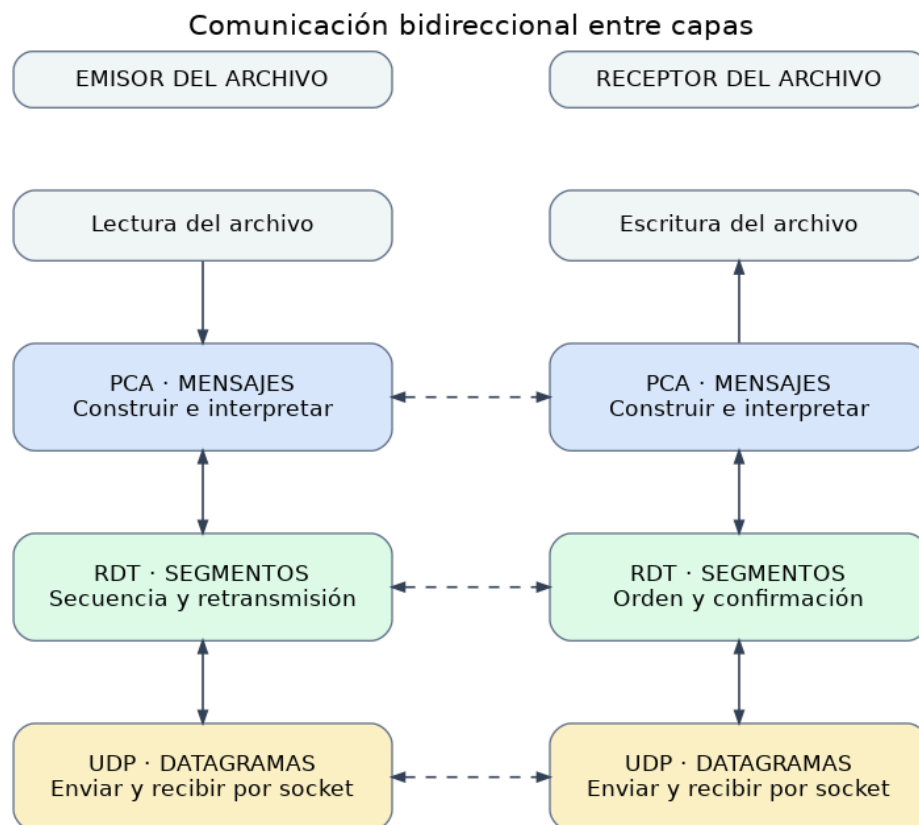
## 3. Implementación

### 3.1. Arquitectura general y organización en capas

La aplicación permite transferir archivos entre un cliente y un servidor mediante operaciones de subida y descarga. En una subida, el cliente es el emisor del archivo y el servidor es el receptor; en una descarga, estos roles se invierten. Ambos extremos utilizan la misma organización en capas para separar el significado de los mensajes de los mecanismos necesarios para transportarlos.

La comunicación se organiza en tres capas: PCA, que define el protocolo de capa de aplicación; RDT, que proporciona transferencia confiable mediante Stop & Wait o SACK; y UDP, que encapsula el acceso a la red mediante sockets. El manejo de archivos acompaña a la lógica de aplicación y se ocupa de la lectura, la escritura y la publicación de los archivos recibidos.

En el emisor, la aplicación lee un bloque y PCA lo representa como un mensaje. RDT agrega la información de secuencia necesaria para confirmar su recepción y retransmitirlo si se pierde. Finalmente, el enlace UDP envía el segmento como un datagrama. En el receptor, el recorrido es inverso: UDP recibe el datagrama, RDT recupera el orden y evita entregar duplicados, y PCA interpreta el mensaje para que la aplicación escriba su contenido o procese una solicitud o respuesta.



## 3.2. Capa PCA: protocolo capa de aplicación

### 3.2.1. Formato y tipos de mensajes

Antes de operar sobre un archivo se acuerda el canal de comunicación, como se explica en la sección 3.5.3. A partir de ese momento, el protocolo de aplicación acuerda la operación, transporta el contenido del archivo y comunica su finalización o rechazo. Cada mensaje completo viaja dentro de un segmento DATOS del canal RDT. La confiabilidad alcanza tanto a los bloques de archivo como a las solicitudes y respuestas.

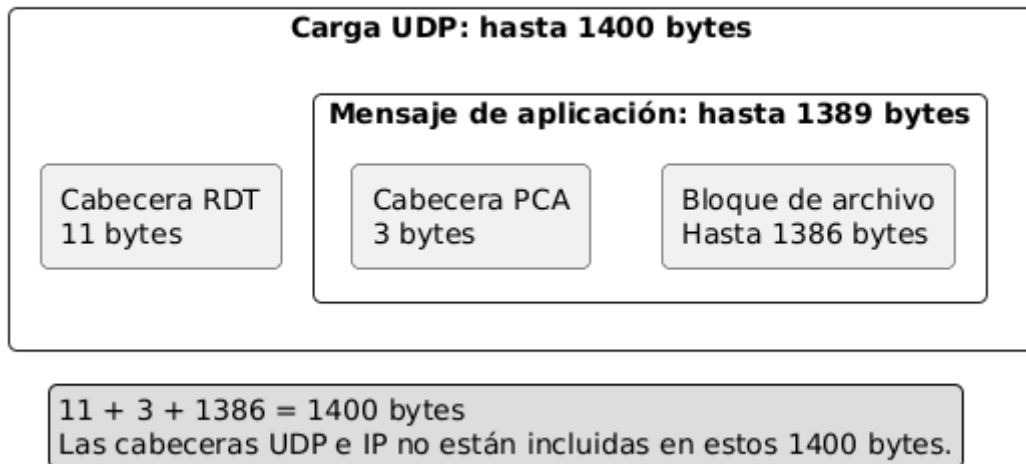
La cabecera de aplicación tiene **3 bytes**, también en orden de red:

| Campo    | Tamaño           | Significado                        |
|----------|------------------|------------------------------------|
| Tipo     | 1 byte           | Tipo de mensaje de aplicación.     |
| Longitud | 2 bytes          | Cantidad de bytes de la carga.     |
| Carga    | Hasta 1386 bytes | Metadatos o contenido del archivo. |

| Mensaje            | Valor | Carga                               | Emisor                                 |
|--------------------|-------|-------------------------------------|----------------------------------------|
| SOLICITUD_SUBIDA   | 1     | Nombre y tamaño del archivo. ▾      | Cliente ▾                              |
| SOLICITUD_DESCARGA | 2     | Nombre del archivo. ▾               | Cliente ▾                              |
| ACEPTADO           | 3     | Vacía para subida; tamaño pa... ▾   | Servidor ▾                             |
| BLOQUE_ARCHIVO     | 4     | Hasta 1386 bytes del archivo. ▾     | Emisor del archivo ▾                   |
| FIN_ARCHIVO        | 5     | Vacía. ▾                            | Emisor del archivo ▾                   |
| COMPLETADO         | 6     | Vacía. ▾                            | Receptor del archivo ▾                 |
| ERROR              | 7     | Código y detalle del error en ... ▾ | Servidor, en los flujos implement... ▾ |

El tamaño máximo de un bloque se obtiene descontando ambas cabeceras de los 1400 bytes disponibles como carga UDP

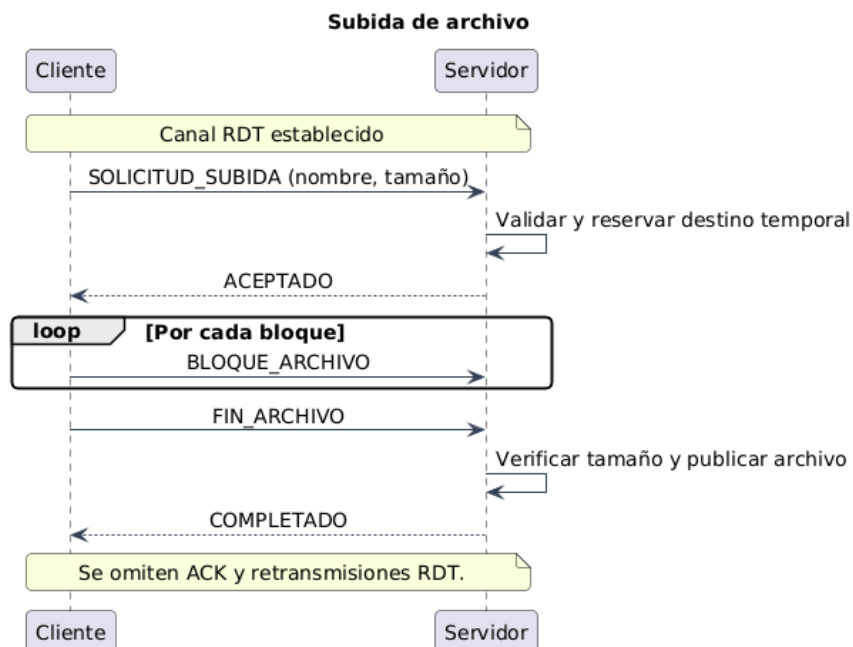
### Encapsulación de un bloque de archivo



#### 3.2.2. Subida de archivos

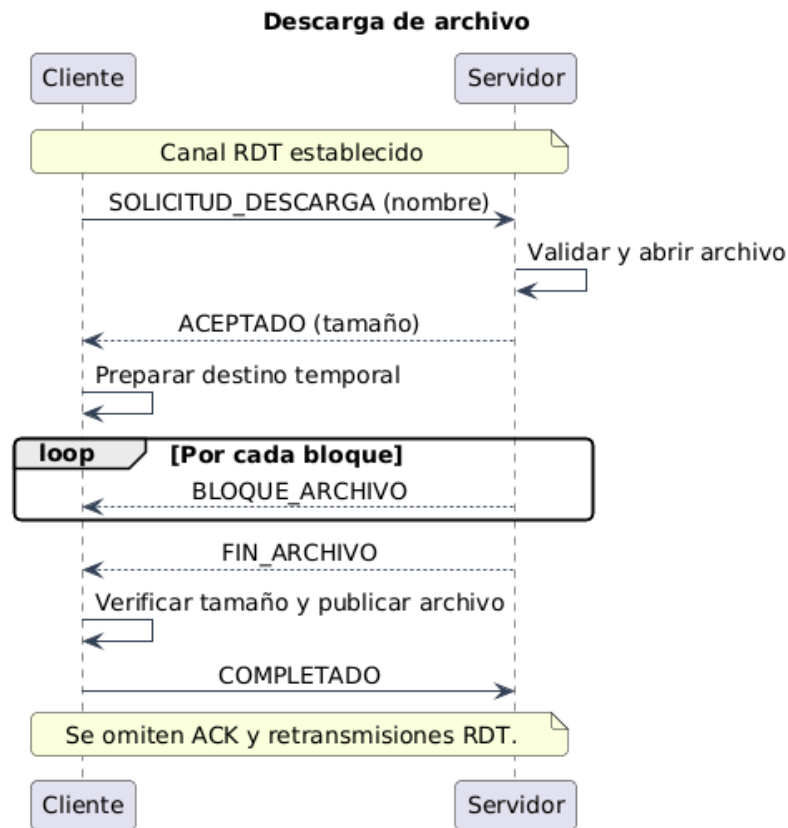
El cliente anuncia el nombre de destino y el tamaño del archivo. El servidor valida el nombre, comprueba que no esté ocupado por un archivo o por otra subida y prepara un temporal antes de responder ACEPTADO. Luego recibe los bloques y controla que la cantidad de bytes no supere el tamaño anunciado.

Al recibir FIN\_ARCHIVO, verifica que el tamaño coincida exactamente, publica el archivo con su nombre definitivo y envía COMPLETADO. El cliente considera exitosa la subida al recibir esta respuesta.



### 3.2.3. Descarga de archivos

El cliente solicita un archivo por su nombre. El servidor comprueba que pueda abrirlo y responde ACEPTADO con su tamaño. A continuación envía los bloques y FIN\_ARCHIVO. El cliente escribe en un temporal, verifica la cantidad recibida y publica el archivo antes de responder COMPLETADO.



### 3.2.4. Condiciones de error

PCA define el mensaje **ERROR** para comunicar el rechazo o la interrupción de una operación. Cuando el cliente recibe esta respuesta, la interpreta como un error remoto e interrumpe la operación.

| Código                    | Condición                                                                                                                                      |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| ErrorNombreArchivo        | El nombre es inválido: está vacío, contiene separadores de ruta o un carácter nulo, es . o .., termina en .tmp o apunta a un enlace simbólico. |
| ErrorArchivolnexistente   | El archivo solicitado no existe o no es un archivo regular.                                                                                    |
| ErrorArchivoExistente     | El destino ya existe; no se permite sobrescribirlo.                                                                                            |
| ErrorTransferenciaEnCurso | Otro cliente está subiendo un archivo con el mismo nombre.                                                                                     |
| ErrorTemporalExistente    | Ya existe un archivo temporal para ese destino, por ejemplo tras una transferencia interrumpida.                                               |

| Código                       | Condición                                                                     |
|------------------------------|-------------------------------------------------------------------------------|
| ErrorLecturaArchivo          | Falló la apertura, lectura o cierre del archivo de origen.                    |
| ErrorEscrituraArchivo        | Falló la creación, escritura, confirmación o limpieza del archivo de destino. |
| ErrorMensaje                 | El mensaje tiene un tamaño, formato, tipo o contenido inválido.               |
| ErrorRespuesta               | El mensaje recibido no corresponde al esperado en ese punto de la operación.  |
| ErrorTransferencialncompleta | La cantidad de bytes recibidos no coincide con el tamaño anunciado.           |

### 3.3. Capa RDT: transferencia confiable

Esta capa transporta los mensajes de PCA sobre UDP y resuelve las pérdidas, los duplicados y la entrega en orden. Sus dos implementaciones comparten una interfaz para enviar, recibir, esperar la confirmación de los envíos pendientes y cerrar el canal. Cada extremo mantiene números de secuencia independientes para los datos que envía y recibe.

#### 3.3.1. Formato del segmento

Cada segmento RDT ocupa como máximo 1400 bytes y se transporta como carga de un datagrama UDP. Su cabecera tiene 11 bytes.

| Campo        | Tamaño             | Significado                                                             |
|--------------|--------------------|-------------------------------------------------------------------------|
| Tipo         | 1 byte ▾           | Identifica SYN, ACK, DATOS o FIN.                                       |
| Secuencia    | 4 bytes ▾          | Identifica un segmento de datos en el sentido de envío correspondiente. |
| Confirmación | 4 bytes ▾          | Indica el próximo segmento esperado en orden.                           |
| Longitud     | 2 bytes ▾          | Cantidad de bytes de la carga.                                          |
| Carga        | Hasta 1389 bytes ▾ | Contenido según el tipo de segmento.                                    |

| Tipo | Valor | Uso en la implementación                                |
|------|-------|---------------------------------------------------------|
| SYN  | 1     | Solicitar y aceptar una sesión, indicando el protocolo. |

| Tipo  | Valor | Uso en la implementación                                        |
|-------|-------|-----------------------------------------------------------------|
| ACK   | 2     | Confirmar segmentos; con SACK también informa rangos recibidos. |
| DATOS | 3     | Transportar un mensaje completo del protocolo de aplicación.    |
| FIN   | 4     | cierre de Stop & Wait y SACK.                                   |

Los números de secuencia cuentan segmentos, no bytes, y comienzan en cero de forma independiente en cada sentido. Una confirmación acumulativa con valor N reconoce los segmentos anteriores a N. Antes de procesar un segmento se comprueba su tamaño, la correspondencia entre la longitud declarada y la carga recibida y que el tipo esté definido. Estas verificaciones controlan el formato, pero no autentican al remitente.

### 3.4. Stop & Wait

En nuestra implementación de este protocolo:

- Se envía un único segmento sin confirmar y se espera su acuse de recibo.
- Cada extremo de la transmisión tiene un conteo de secuencia independiente y lo van avanzando de a uno en uno.
- El ACK no lleva carga pero lleva el próximo número de secuencia esperado. En caso de un envío duplicado se vuelve a confirmar para evitar retransmisiones innecesarias.

El RTO no es un valor fijo, comienza en 1 segundo, el valor luego se actualiza con la siguiente fórmula:

$$RTO = SRTT + 4 \times DEVRTT$$

Iniciando como:

$$SRTT = \text{muestra}$$

$$DEVRTT = \text{muestra} / 2$$

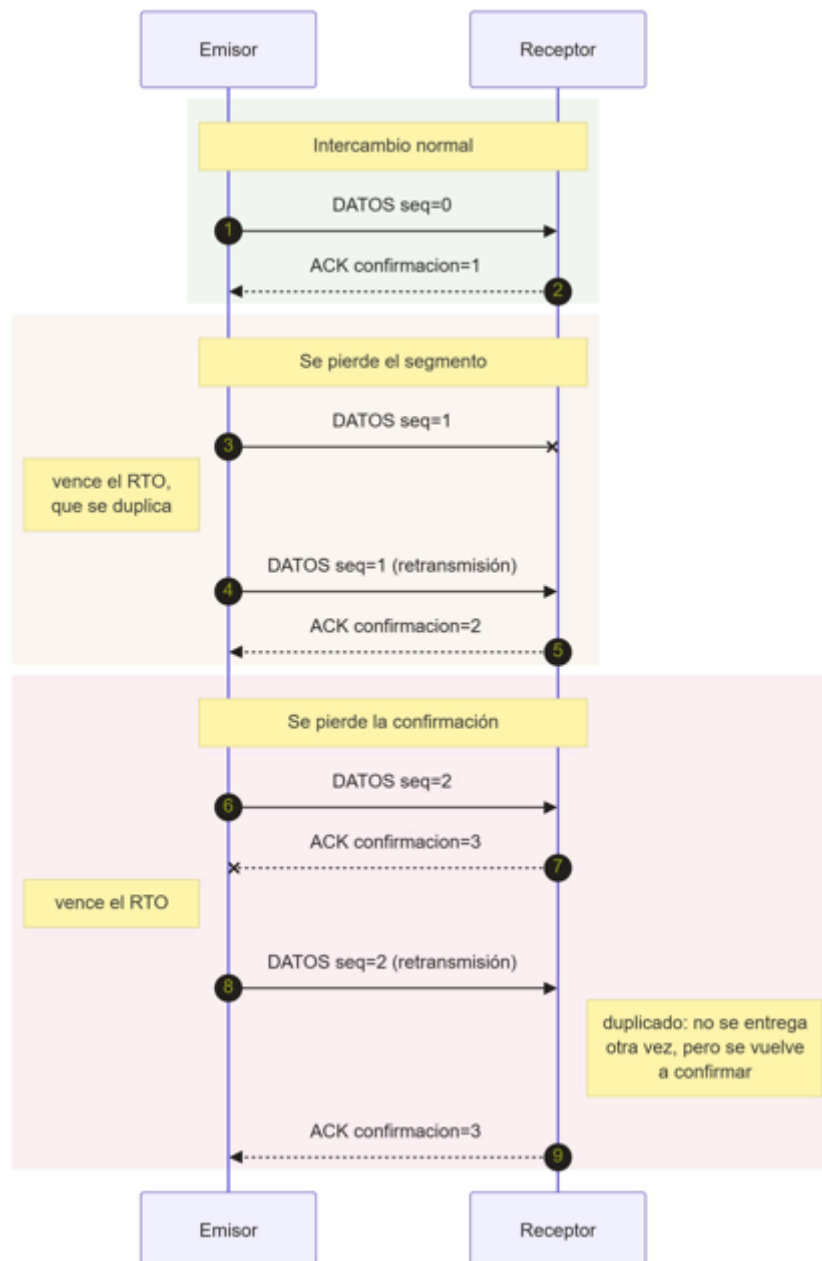
Y calculándose luego como:

$$DEVRTT = (1 - \beta) \cdot DEVRTT + \beta \cdot |\text{muestra} - SRTT| \quad \beta = \frac{1}{4}$$

$$SRTT = (1 - \alpha) \cdot SRTT + \alpha \cdot \text{muestra} \quad \alpha = \frac{1}{8}$$

Se ignora el RTT de la retransmisiones porque no se sabe a cual de los envíos corresponde.

- El único caso en el que hay una retransmisión es al vencerse el timeout, el RTO se duplica en cada timeout (hasta 8 segundos) y tras 10 intentos el canal se corta con un error.



### 3.5. SACK

En este caso, a diferencia de Stop and Wait, se mantienen varios segmentos sin confirmar en simultáneo, esto está definido por el ancho de la ventana (en nuestro caso 64) y el receptor informa de manera selectiva que fue lo que recibió fuera de orden, de modo de que solo se retransmita lo que falte.

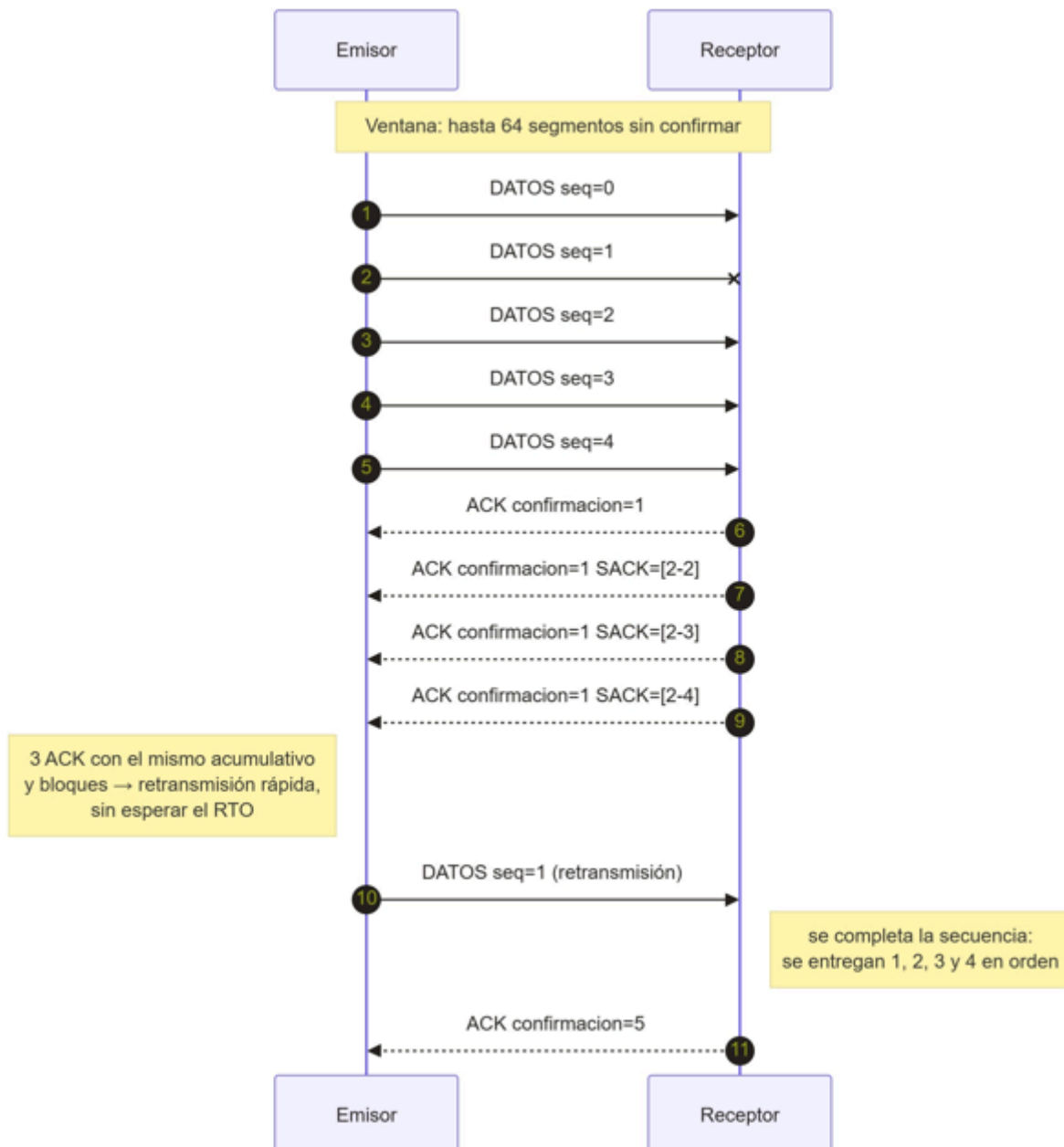
Al igual que en Stop and Wait el RTO no es un valor fijo, se recalcula en base a los mensajes recibidos siguiendo la misma estrategia.

La retransmisión puede darse por dos circunstancias:

- Timeout, reenviando cada uno de los segmentos cuyo temporizador haya vencido.
- ACK duplicados (fast retransmit), tras tres ACK con el mismo acumulativo se reenvía el segmento faltante sin esperar el timeout.

El ACK lleva dos cosas a la vez:

- Confirmación acumulativo de la cabecera
- En la carga la lista de rangos ya recibidos por encima del acumulativo (ej: Si el receptor esperaba el 5, no le llegó, pero recibió del 6 al 9 y del 11 al 12, manda confirmacion=5 con los rangos [(6,9), (11,12)]. El emisor libera todo lo anterior al 5 más esos dos bloques, y queda sabiendo que debe reenviar el 5 y el 10, y solo esos.)



### 3.6. Capa UDP y uso de la API de sockets

La capa UDP ofrece a RDT una interfaz común con dos operaciones: enviar bytes al otro extremo y recibir un datagrama con un tiempo de espera. Esta interfaz tiene dos implementaciones porque el cliente y el servidor acceden de distinta manera a los datos recibidos.

En el cliente, cada transferencia tiene su propio socket UDP. El enlace puede leer directamente de él mediante `recvfrom()` y verificar que el datagrama provenga del servidor configurado. Como ese socket pertenece a una única operación, establecer su timeout o cerrarlo al terminar no afecta a otras transferencias.

En el servidor, todas las transferencias comparten un único socket. Si cada hilo ejecutara `recvfrom()` sobre él, cualquiera podría consumir el próximo datagrama disponible, aunque perteneciera a otro cliente. Filtrar por dirección después de leer no resolvería el problema: el datagrama ya habría sido retirado del socket y no llegaría al hilo que lo esperaba.

Por este motivo, el enlace utilizado por cada transferencia del servidor recibe desde una cola propia, alimentada por un receptor central que distribuye los datagramas según su dirección de origen. Solo ese receptor lee del socket compartido. Los envíos sí se realizan directamente mediante `sendto()`, ya que cada enlace conserva la dirección del cliente al que debe responder. La organización del despachador y las sesiones se desarrolla en la sección 3.7.

| Aspecto                        | Enlace del cliente                                | Enlace de una transferencia del servidor                 |
|--------------------------------|---------------------------------------------------|----------------------------------------------------------|
| Socket utilizado               | Exclusivo de la transferencia.                    | Compartido por todas las transferencias.                 |
| Origen de los datos al recibir | Lectura directa con <code>recvfrom()</code> .     | Cola propia, alimentada por el despachador.              |
| Selección del interlocutor     | Comprueba que el origen sea el servidor esperado. | El despachador selecciona la cola según IP y puerto.     |
| Envío                          | <code>sendto()</code> hacia el servidor.          | <code>sendto()</code> hacia el cliente correspondiente.  |
| Tiempo de espera de RDT        | Se aplica a la recepción del socket propio.       | Se aplica a la espera en la cola propia.                 |
| Finalización                   | El cliente cierra su socket al terminar.          | Se interrumpe el enlace sin cerrar el socket compartido. |

La separación también permite administrar tiempos de espera independientes. Si cada hilo del servidor modificara el timeout del socket compartido, afectaría la recepción de las demás transferencias. Al esperar sobre colas distintas, cada canal RDT utiliza su propio plazo sin modificar el comportamiento del receptor central.

### 3.7. Organización y funcionamiento del servidor

El servidor integra las tres capas para atender transferencias concurrentes sobre un único socket UDP. Comparte la recepción de datagramas y el almacenamiento, pero mantiene por separado el estado de comunicación de cada cliente. Esta separación se organiza mediante sesiones, un despachador y un hilo por transferencia.

#### 3.7.1. Sesiones y concurrencia

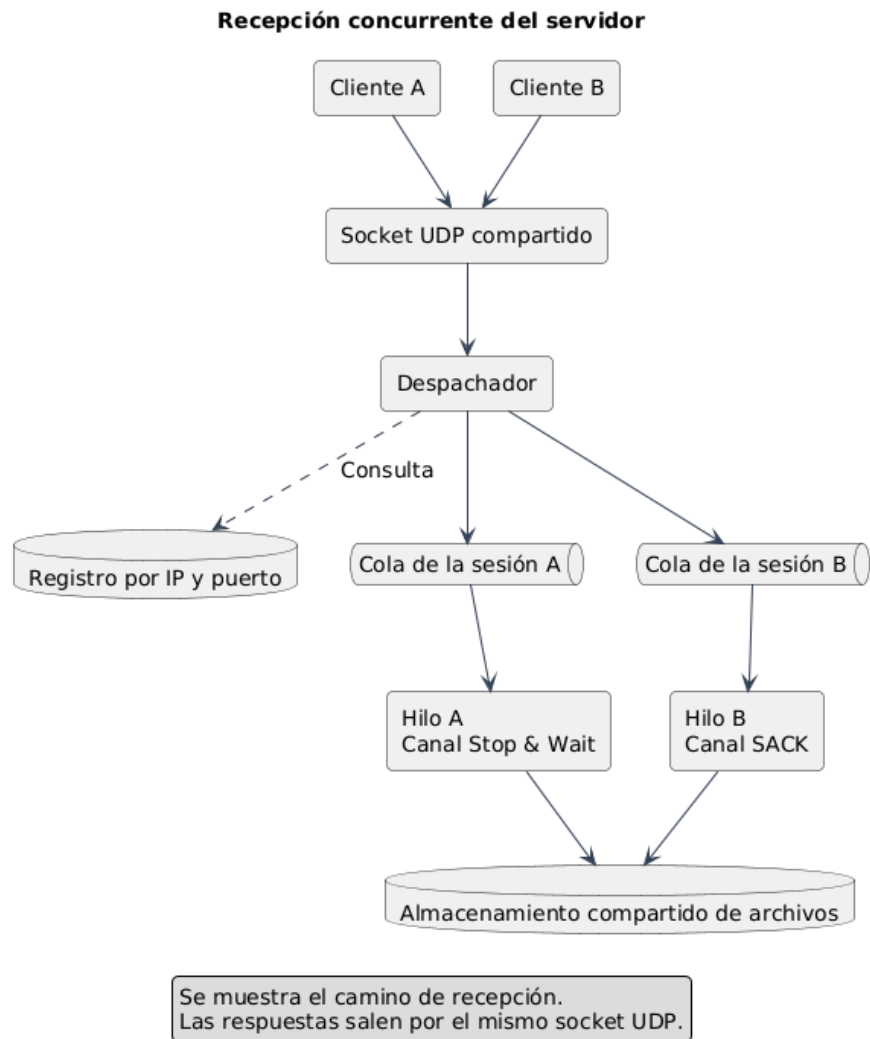
Una sesión es el contexto que el servidor mantiene para atender una transferencia de un cliente. Contiene la dirección del cliente, el protocolo elegido, una cola de recepción, un canal RDT con sus números de secuencia y temporizadores y un hilo que ejecuta la operación solicitada. Cada sesión corresponde a una única subida o descarga.

El servidor identifica las sesiones por el par (IP, puerto) de origen de los datagramas. El cliente crea un socket UDP para cada operación y conserva esa dirección durante la transferencia. Así, varios clientes pueden utilizar simultáneamente protocolos distintos: el estado de Stop & Wait o SACK pertenece a cada sesión, no al socket compartido del servidor.

### **3.7.2. Despachador y distribución de datagramas**

El servidor utiliza un único socket UDP y un despachador que centraliza la recepción. Por cada datagrama, valida el formato del segmento y consulta el registro de sesiones. Si pertenece a una sesión activa, lo coloca en su cola; el hilo correspondiente lo obtiene desde allí y continúa el procesamiento. Las solicitudes de establecimiento reciben un tratamiento específico, explicado en la sección 3.7.3.

Esta organización evita que varios hilos compitan por leer del mismo socket y que uno de ellos consuma datagramas destinados a otra sesión. Los hilos sí comparten el socket para enviar sus respuestas, indicando en cada envío la dirección del cliente correspondiente.

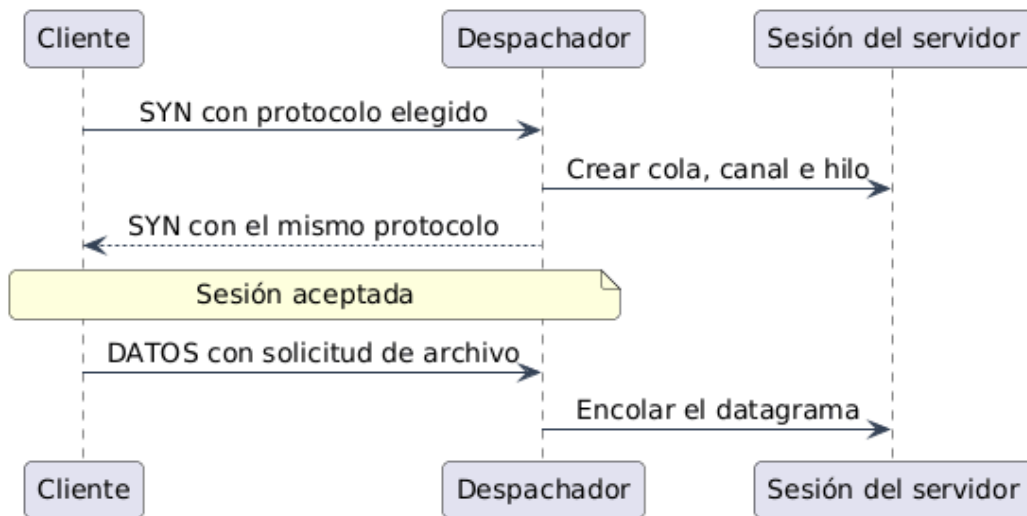


Se admiten hasta **50 sesiones activas**, cada una con una cola de **4096 datagramas**. Si se alcanza el límite de sesiones, se descarta la nueva solicitud y el cliente termina por agotar sus intentos de establecimiento. Si una cola está llena, se descarta el datagrama; el canal confiable deberá recuperarlo mediante retransmisión. Un cerrojo protege el registro de sesiones y otro coordina las reservas de nombres para impedir subidas simultáneas al mismo destino.

### 3.7.3. Establecimiento de sesión

Antes de enviar una solicitud de archivo, el cliente envía un SYN cuya carga contiene un byte con el protocolo elegido: **0 para Stop & Wait** o **1 para SACK**. El servidor valida la selección, registra la sesión, prepara su canal y responde con un SYN que contiene el mismo valor. Luego inicia el hilo encargado de la operación.

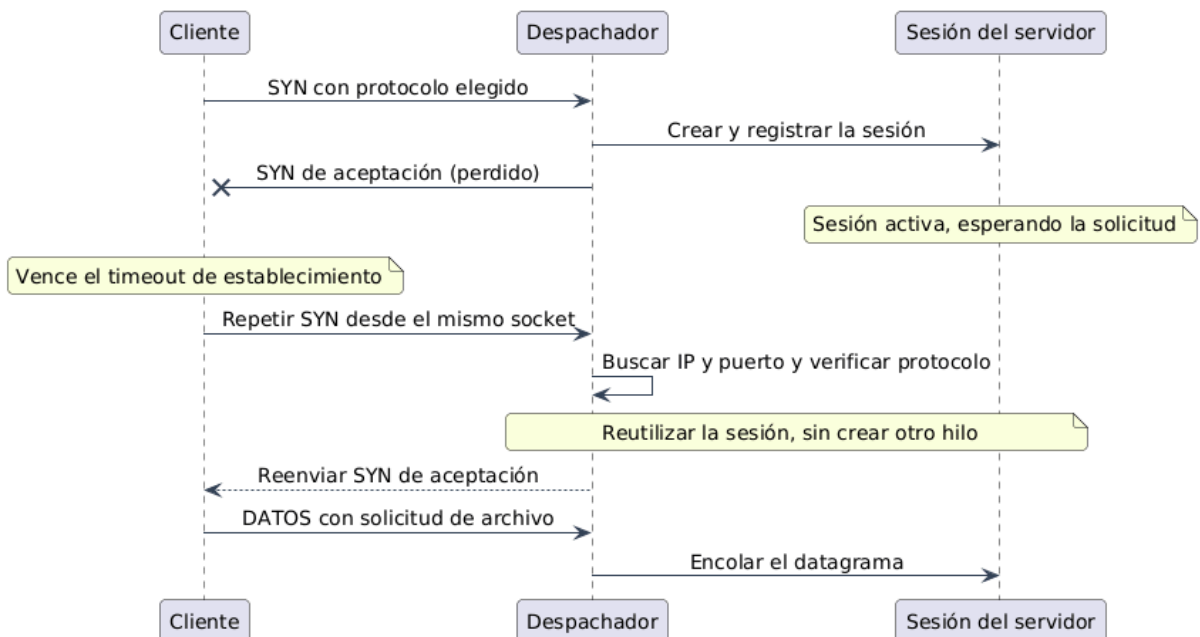
### Establecimiento de sesión



El establecimiento tiene su propio mecanismo de reintentos porque todavía no existe un canal RDT operativo en el cliente. Se realizan hasta **seis envíos del SYN en total**, con una espera inicial de un segundo que se duplica ante un timeout hasta un máximo de ocho segundos.

Si se pierde el SYN de respuesta, el servidor ya tiene una sesión creada, pero el cliente todavía espera la aceptación. Al vencer su timeout, el cliente repite el SYN desde el mismo socket y con el mismo protocolo. El despachador encuentra la sesión activa por su par (IP, puerto) y reenvía la respuesta guardada. No crea otra cola, otro canal ni otro hilo, y tampoco reinicia el estado de la sesión. Cuando el cliente recibe la respuesta, puede enviar la solicitud de archivo.

### Recuperación de un SYN de aceptación perdido

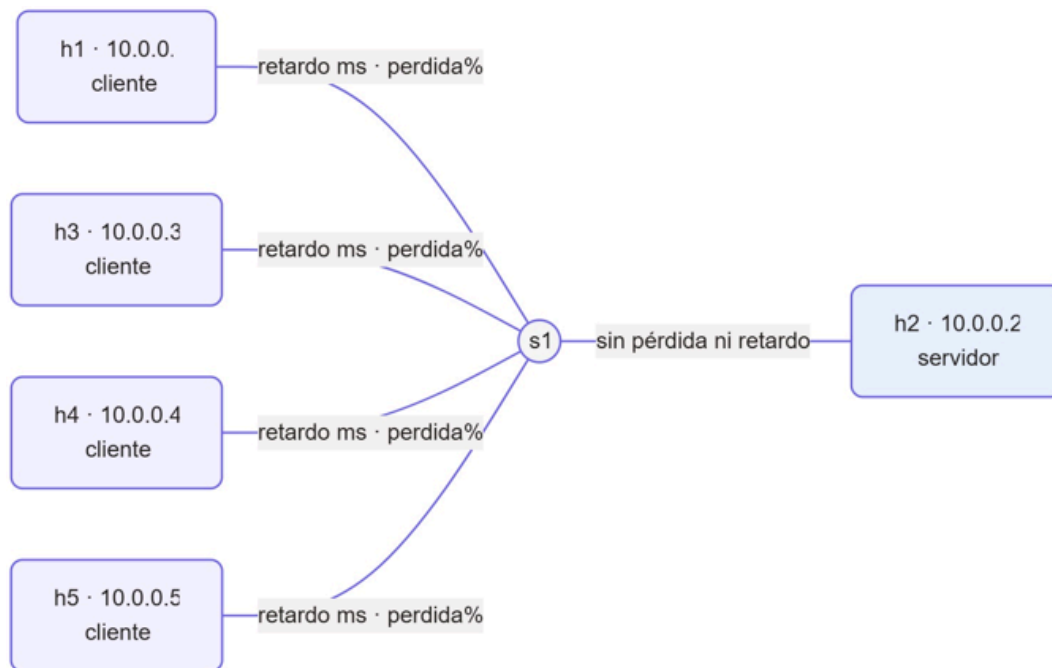


## 4. Pruebas

### 4.1. Entorno de pruebas

Las pruebas se ejecutan sobre Mininet con una topología parametrizada por la pérdida y el RTT.

Cuatro hosts cliente y uno servidor sobre un único switch. Las direcciones se asignan explícitamente para que el servidor sea siempre 10.0.0.2



Se probaron con las siguientes configuraciones:

| Pérdida por sentido | RTT    |
|---------------------|--------|
| 0 %                 | 40 ms  |
| 10 %                | 40 ms  |
| 20 %                | 40 ms  |
| 10 %                | 300 ms |

## 4.2. Casos de prueba

| Dimensión           | Valores                                                        |
|---------------------|----------------------------------------------------------------|
| Tamaños             | 64 KB, 256 KB, 1 MB (comparación) y 5 MB (requisito de tiempo) |
| Protocolos          | Stop & Wait y SACK                                             |
| Direcciones         | subida y descarga                                              |
| Pérdida por sentido | 0 %, 10 % y 20 %                                               |
| RTT                 | 40 ms y 300 ms                                                 |

Los archivos se generan con contenido aleatorio (`os.urandom`), de modo que son binarios sin estructura y sin repeticiones que pudieran enmascarar un error de transferencia.

Se probó la concurrencia con cuatro transferencias simultáneas —dos subidas y dos descargas de 1 MB— desde cuatro hosts cliente distintos contra el mismo servidor. Se verifica la integridad de las cuatro y se contrasta el tiempo total contra la suma de los tiempos individuales.

4.3. Capturas de ejecución y logs del servidor

Prueba de concurrencia:

```
*** Creating network
*** Adding hosts:
h1 h2 h3 h4 h5
*** Adding switches:
s1
*** Adding links:
(150ms delay 10.00000% loss) (150ms delay 10.00000% loss) (h1, s1) (150ms delay 10.00000% loss) (150ms delay 10.00000% loss) (h3, s1) (150ms delay 1
0.00000% loss) (150ms delay 10.00000% loss) (h4, s1) (150ms delay 10.00000% loss) (150ms delay 10.00000% loss) (h5, s1) (s1, h2)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller

*** Starting 1 switches
s1 ... (150ms delay 10.00000% loss) (150ms delay 10.00000% loss) (150ms delay 10.00000% loss) (150ms delay 10.00000% loss)
perdida 10% por sentido, RTT 300 ms
log del servidor en /tmp/servidor-tp.log (se copia a /mnt/c/Users/Jaz05/Desktop/Redes/TPs-REDES/mediciones/servidor.log al terminar)
-- comparacion Stop & Wait contra SACK --
upload sw 64KB 28.39 s 2.3 KB/s ok
download sw 64KB 23.40 s 2.7 KB/s ok
upload sw 256KB 117.12 s 2.2 KB/s ok
download sw 256KB 77.15 s 3.3 KB/s ok
upload sw 1MB 356.22 s 2.9 KB/s ok
download sw 1MB 308.74 s 3.3 KB/s ok
upload sack 64KB 4.73 s 13.5 KB/s ok
download sack 64KB 3.21 s 19.9 KB/s ok
upload sack 256KB 4.11 s 62.2 KB/s ok
download sack 256KB 5.08 s 50.4 KB/s ok
upload sack 1MB 7.07 s 144.8 KB/s ok
download sack 1MB 8.27 s 123.9 KB/s ok
-- requisito: SACK con 5MB en menos de 120 s --
upload sack 5MB 24.14 s 212.1 KB/s ok
download sack 5MB 27.06 s 189.2 KB/s ok
```

Prueba servidor no levantado:

| No. | Time      | Source   | Destination | Protocol | Lengt | Info               |
|-----|-----------|----------|-------------|----------|-------|--------------------|
| 1   | 0.000000  | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54    | SYN protocolo=SACK |
| 2   | 0.000040  | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54    | SYN protocolo=SACK |
| 3   | 0.927517  | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54    | SYN protocolo=SACK |
| 4   | 4.929620  | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54    | SYN protocolo=SACK |
| 5   | 21.439424 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54    | SYN protocolo=SACK |
| 6   | 30.021967 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54    | SYN protocolo=SACK |
| 7   | 31.023455 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54    | SYN protocolo=SACK |
| 8   | 33.025578 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54    | SYN protocolo=SACK |
| 9   | 37.029497 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54    | SYN protocolo=SACK |
| 10  | 45.034535 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54    | SYN protocolo=SACK |
| 11  | 53.532804 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54    | SYN protocolo=SACK |

Subida de archivo:

| No. | Time     | Source   | Destination | Protocol | Length | Info                                                                |
|-----|----------|----------|-------------|----------|--------|---------------------------------------------------------------------|
| 1   | 0.000000 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54     | SYN protocolo=SACK                                                  |
| 2   | 0.021114 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 54     | SYN protocolo=SACK                                                  |
| 3   | 0.041686 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 95     | DATOS seq=0 SOLICITUD_SUBIDA {"nombre": "cap.bin", "tamano": 65536} |
| 4   | 0.062154 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 53     | ACK confirmacion=1                                                  |
| 5   | 0.062809 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 56     | DATOS seq=0 ACEPTADO                                                |
| 6   | 0.083031 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 53     | ACK confirmacion=1                                                  |
| 7   | 0.083250 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=1 BLOQUE_ARCHIVO 1386 bytes                               |
| 8   | 0.083305 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=2 BLOQUE_ARCHIVO 1386 bytes                               |
| 9   | 0.083437 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=3 BLOQUE_ARCHIVO 1386 bytes                               |
| 10  | 0.083493 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=4 BLOQUE_ARCHIVO 1386 bytes                               |
| 11  | 0.083661 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=5 BLOQUE_ARCHIVO 1386 bytes                               |
| 12  | 0.083665 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=6 BLOQUE_ARCHIVO 1386 bytes                               |
| 13  | 0.083768 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=7 BLOQUE_ARCHIVO 1386 bytes                               |
| 14  | 0.083846 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=8 BLOQUE_ARCHIVO 1386 bytes                               |
| 15  | 0.084046 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=9 BLOQUE_ARCHIVO 1386 bytes                               |
| 16  | 0.084102 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=10 BLOQUE_ARCHIVO 1386 bytes                              |
| 17  | 0.084189 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=11 BLOQUE_ARCHIVO 1386 bytes                              |
| 18  | 0.084279 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=12 BLOQUE_ARCHIVO 1386 bytes                              |
| 19  | 0.084519 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=13 BLOQUE_ARCHIVO 1386 bytes                              |
| 20  | 0.084707 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=15 BLOQUE_ARCHIVO 1386 bytes                              |
| 21  | 0.084792 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=16 BLOQUE_ARCHIVO 1386 bytes                              |
| 22  | 0.085003 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=17 BLOQUE_ARCHIVO 1386 bytes                              |
| 23  | 0.085101 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=18 BLOQUE_ARCHIVO 1386 bytes                              |
| 24  | 0.085282 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=20 BLOQUE_ARCHIVO 1386 bytes                              |
| 25  | 0.085436 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=21 BLOQUE_ARCHIVO 1386 bytes                              |
| 26  | 0.085535 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 1442   | DATOS seq=22 BLOQUE_ARCHIVO 1386 bytes                              |

Retransmisión de segmento faltante 19:

| Time  | Source   | Destination | Protocol | Length | Info                                   |
|-------|----------|-------------|----------|--------|----------------------------------------|
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 69     | ACK confirmacion=14 SACK=[15-18,20-41] |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 69     | ACK confirmacion=14 SACK=[15-18,20-42] |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 69     | ACK confirmacion=14 SACK=[15-18,20-44] |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 69     | ACK confirmacion=14 SACK=[15-18,20-45] |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 69     | ACK confirmacion=14 SACK=[15-18,20-46] |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 69     | ACK confirmacion=14 SACK=[15-18,20-47] |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 69     | ACK confirmacion=14 SACK=[15-18,20-48] |
| 0.0.1 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=14 BLOQUE_ARCHIVO 1386 bytes |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 69     | ACK confirmacion=15 SACK=[15-18,20-48] |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 69     | ACK confirmacion=16 SACK=[16-18,20-48] |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 69     | ACK confirmacion=17 SACK=[17-18,20-48] |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 69     | ACK confirmacion=18 SACK=[18-18,20-48] |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=19 SACK=[20-48]       |
| 0.0.1 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=19 BLOQUE_ARCHIVO 1386 bytes |
| 0.0.1 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 56     | DATOS seq=49 FIN_ARCHIVO               |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=20 SACK=[20-48]       |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=21 SACK=[21-48]       |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=22 SACK=[22-48]       |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=23 SACK=[23-48]       |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=24 SACK=[24-48]       |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=25 SACK=[25-48]       |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=27 SACK=[27-48]       |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=28 SACK=[28-48]       |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=29 SACK=[29-48]       |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=30 SACK=[30-48]       |
| 0.0.2 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 61     | ACK confirmacion=31 SACK=[31-48]       |

Fin de transmision:

|     |          |          |          |       |    |                                  |
|-----|----------|----------|----------|-------|----|----------------------------------|
| 116 | 0.268961 | 10.0.0.2 | 10.0.0.1 | TP-FT | 61 | ACK confirmacion=37 SACK=[37-48] |
| 117 | 0.269047 | 10.0.0.2 | 10.0.0.1 | TP-FT | 61 | ACK confirmacion=38 SACK=[38-48] |
| 118 | 0.269158 | 10.0.0.2 | 10.0.0.1 | TP-FT | 61 | ACK confirmacion=39 SACK=[39-48] |
| 119 | 0.269217 | 10.0.0.2 | 10.0.0.1 | TP-FT | 61 | ACK confirmacion=40 SACK=[40-48] |
| 120 | 0.269416 | 10.0.0.2 | 10.0.0.1 | TP-FT | 61 | ACK confirmacion=42 SACK=[42-48] |
| 121 | 0.269419 | 10.0.0.2 | 10.0.0.1 | TP-FT | 61 | ACK confirmacion=43 SACK=[43-48] |
| 122 | 0.269518 | 10.0.0.2 | 10.0.0.1 | TP-FT | 61 | ACK confirmacion=44 SACK=[44-48] |
| 123 | 0.269587 | 10.0.0.2 | 10.0.0.1 | TP-FT | 61 | ACK confirmacion=45 SACK=[45-48] |
| 124 | 0.269652 | 10.0.0.2 | 10.0.0.1 | TP-FT | 61 | ACK confirmacion=46 SACK=[46-48] |
| 125 | 0.269722 | 10.0.0.2 | 10.0.0.1 | TP-FT | 61 | ACK confirmacion=47 SACK=[47-48] |
| 126 | 0.269789 | 10.0.0.2 | 10.0.0.1 | TP-FT | 61 | ACK confirmacion=48 SACK=[48-48] |
| 127 | 0.269864 | 10.0.0.2 | 10.0.0.1 | TP-FT | 53 | ACK confirmacion=49              |
| 128 | 0.269962 | 10.0.0.2 | 10.0.0.1 | TP-FT | 53 | ACK confirmacion=50              |
| 129 | 0.270169 | 10.0.0.2 | 10.0.0.1 | TP-FT | 56 | DATOS seq=1 COMPLETADO           |
| 130 | 0.290374 | 10.0.0.1 | 10.0.0.2 | TP-FT | 53 | ACK confirmacion=2               |

Inicio de descarga:

| No. | Time     | Source   | Destination | Protocol | Length | Info                                                 |
|-----|----------|----------|-------------|----------|--------|------------------------------------------------------|
| 1   | 0.000000 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 54     | SYN protocolo=SACK                                   |
| 2   | 0.024458 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 54     | SYN protocolo=SACK                                   |
| 3   | 0.044745 | 10.0.0.1 | 10.0.0.2    | TP-FT    | 77     | DATOS seq=0 SOLICITUD_DESCARGA {"nombre": "cap.bin"} |
| 4   | 0.066730 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 74     | DATOS seq=0 ACEPTADO {"tamaño": 65536}               |
| 5   | 0.066911 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=1 BLOQUE_ARCHIVO 1386 bytes                |
| 6   | 0.066952 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=2 BLOQUE_ARCHIVO 1386 bytes                |
| 7   | 0.067058 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=3 BLOQUE_ARCHIVO 1386 bytes                |
| 8   | 0.067149 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=4 BLOQUE_ARCHIVO 1386 bytes                |
| 9   | 0.067244 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=5 BLOQUE_ARCHIVO 1386 bytes                |
| 10  | 0.067340 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=6 BLOQUE_ARCHIVO 1386 bytes                |
| 11  | 0.067429 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=7 BLOQUE_ARCHIVO 1386 bytes                |
| 12  | 0.067738 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=9 BLOQUE_ARCHIVO 1386 bytes                |
| 13  | 0.067797 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=10 BLOQUE_ARCHIVO 1386 bytes               |
| 14  | 0.068153 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=12 BLOQUE_ARCHIVO 1386 bytes               |
| 15  | 0.068250 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=14 BLOQUE_ARCHIVO 1386 bytes               |
| 16  | 0.068336 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=15 BLOQUE_ARCHIVO 1386 bytes               |
| 17  | 0.068453 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=16 BLOQUE_ARCHIVO 1386 bytes               |
| 18  | 0.068546 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=17 BLOQUE_ARCHIVO 1386 bytes               |
| 19  | 0.068753 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=18 BLOQUE_ARCHIVO 1386 bytes               |
| 20  | 0.068799 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=19 BLOQUE_ARCHIVO 1386 bytes               |
| 21  | 0.068952 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=20 BLOQUE_ARCHIVO 1386 bytes               |
| 22  | 0.069000 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=21 BLOQUE_ARCHIVO 1386 bytes               |
| 23  | 0.069144 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=22 BLOQUE_ARCHIVO 1386 bytes               |
| 24  | 0.069284 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=24 BLOQUE_ARCHIVO 1386 bytes               |
| 25  | 0.069453 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=25 BLOQUE_ARCHIVO 1386 bytes               |
| 26  | 0.069507 | 10.0.0.2 | 10.0.0.1    | TP-FT    | 1442   | DATOS seq=26 BLOQUE_ARCHIVO 1386 bytes               |

> Frame 1: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0

> Ethernet II, Src: e2:d7:4d:53:88:75 (e2:d7:4d:53:88:75), Dst: ba:db:72:70:81:d1 (ba:db:72:70:81:d1)

> Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2

> User Datagram Protocol, Src Port: 44979, Dst Port: 8080

TP Redes - File Transfer, SYN

Tipo: SYN (1)

0000

ba db 72 70 81 d1 e2 d7 4d 53 88 75 08 00 45 00

0010

00 28 47 db 40 00 40 11 de e7 0a 00 00 01 0a 00

0020

00 02 af b3 1f 90 00 14 14 28 01 00 00 00 00 00

0030

00 00 00 00 01 01

.....rp.....MS-u..E

.....(6@.....

.....(.....

.....

Retransmisión de 46 y fin:

|     |          |          |          |       |      |                                        |
|-----|----------|----------|----------|-------|------|----------------------------------------|
| 117 | 0.434993 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=33 SACK=[33-45,47-49] |
| 118 | 0.434994 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=34 SACK=[34-45,47-49] |
| 119 | 0.434995 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=35 SACK=[35-45,47-49] |
| 120 | 0.434996 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=36 SACK=[36-45,47-49] |
| 121 | 0.434996 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=37 SACK=[37-45,47-49] |
| 122 | 0.435040 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=38 SACK=[38-45,47-49] |
| 123 | 0.435042 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=39 SACK=[39-45,47-49] |
| 124 | 0.435043 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=40 SACK=[40-45,47-49] |
| 125 | 0.435044 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=41 SACK=[41-45,47-49] |
| 126 | 0.435056 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=42 SACK=[42-45,47-49] |
| 127 | 0.435096 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=43 SACK=[43-45,47-49] |
| 128 | 0.435100 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=44 SACK=[44-45,47-49] |
| 129 | 0.435112 | 10.0.0.1 | 10.0.0.2 | TP-FT | 69   | ACK confirmacion=45 SACK=[45-45,47-49] |
| 130 | 0.435140 | 10.0.0.1 | 10.0.0.2 | TP-FT | 61   | ACK confirmacion=46 SACK=[47-49]       |
| 131 | 0.456551 | 10.0.0.2 | 10.0.0.1 | TP-FT | 1442 | DATOS seq=46 BLOQUE_ARCHIVO 1386 bytes |
| 132 | 0.476900 | 10.0.0.1 | 10.0.0.2 | TP-FT | 61   | ACK confirmacion=47 SACK=[47-49]       |
| 133 | 0.476909 | 10.0.0.1 | 10.0.0.2 | TP-FT | 61   | ACK confirmacion=48 SACK=[48-49]       |
| 134 | 0.476910 | 10.0.0.1 | 10.0.0.2 | TP-FT | 61   | ACK confirmacion=49 SACK=[49-49]       |
| 135 | 0.476911 | 10.0.0.1 | 10.0.0.2 | TP-FT | 53   | ACK confirmacion=50                    |
| 136 | 0.477100 | 10.0.0.1 | 10.0.0.2 | TP-FT | 56   | DATOS seq=1 COMPLETADO                 |
| 137 | 0.500223 | 10.0.0.2 | 10.0.0.1 | TP-FT | 53   | ACK confirmacion=2                     |

#### 4.4. Análisis: Stop & Wait contra SACK

Con solo un emisor y un receptor:

| Protocolo   | Tamaño de archivo | Pérdida (%) | RTT (ms) | Tiempo (s) | Throughput promedio (KB/s) |
|-------------|-------------------|-------------|----------|------------|----------------------------|
| Stop & Wait | 64 kb             | 10          | 300      | 28,39      | 2,3                        |
| Stop & Wait | 256 kb            | 10          | 300      | 117,12     | 2,2                        |
| Stop & Wait | 1 mb              | 10          | 300      | 356,22     | 2,9                        |
| Stop & Wait | 1 mb              | 0           | 40       | 32,51      | 31,5                       |
| Stop & Wait | 1 mb              | 10          | 40       | 50,18      | 20,4                       |
| Stop & Wait | 1 mb              | 20          | 40       | 209,62     | 4,9                        |
| SACK        | 64 kb             | 10          | 300      | 4,73       | 13,5                       |
| SACK        | 256 kb            | 10          | 300      | 4,11       | 62,2                       |
| SACK        | 1 mb              | 10          | 300      | 7,07       | 144,8                      |
| SACK        | 1 mb              | 0           | 40       | 1,80       | 568,8                      |
| SACK        | 1 mb              | 10          | 40       | 2,17       | 472,4                      |
| SACK        | 1 mb              | 20          | 40       | 3,25       | 314,9                      |
| SACK        | 5 mb              | 10          | 40       | 10.50      | 487,7                      |

Pruebas con concurrencia:

Cuatro transferencias de 1 MB con SACK, 10 % de pérdida por sentido, una por host cliente:

RTT 40 ms

| Host | Operación | Tiempo (s) | Throughput (KB/s) |
|------|-----------|------------|-------------------|
| h1   | subida    | 10.82      | 94.6              |
| h3   | subida    | 10.78      | 95.0              |
| h4   | descarga  | 12.83      | 79.8              |
| h5   | descarga  | 10.39      | 98.6              |

RTT 300 ms

| Host | Operación | Tiempo (s) | Throughput (KB/s) |
|------|-----------|------------|-------------------|
| h1   | subida    | 8.81       | 116.2             |
| h3   | subida    | 9.95       | 102.9             |
| h4   | descarga  | 9.07       | 112.9             |
| h5   | descarga  | 9.11       | 112.4             |

## 5. Preguntas a responder

### 5.1. Describa la arquitectura Cliente-Servidor.

La arquitectura cliente-servidor es un modelo de organización distribuida en el que un servidor ofrece un servicio a múltiples clientes. Existe una asimetría de roles: el cliente inicia la comunicación para solicitar un servicio y el servidor recibe la solicitud, la procesa y responde. Ambos utilizan un protocolo común que define cómo intercambiar e interpretar los mensajes.

En el esquema utilizado en este trabajo, los clientes conocen la dirección del servidor y se comunican con él para subir o descargar archivos, sin intercambiar información directamente entre sí. L

### 5.2. ¿Cuál es la función de un protocolo de capa de aplicación?

La capa de aplicación permite que procesos ejecutados en distintos hosts intercambien información a través de una red. Un protocolo de esta capa establece las reglas que hacen posible esa comunicación: define los tipos de mensajes, su sintaxis o formato, su semántica y las condiciones bajo las cuales deben enviarse y responderse.

Sus responsabilidades dependen del servicio que ofrece. Por ejemplo, HTTP define intercambios para acceder a recursos web, SMTP permite transferir correo electrónico y DNS permite consultar información asociada a nombres de dominio. El protocolo de aplicación utiliza los servicios de la capa de transporte y, cuando estos no cubren sus necesidades, puede incorporar mecanismos adicionales. En este trabajo, la lógica de subida y descarga se apoya en una capa de transferencia confiable sobre UDP que resuelve las pérdidas, las retransmisiones y el orden de entrega.

### 5.3. Detalle el protocolo de aplicación desarrollado en este trabajo.

El protocolo de aplicación se describe en la sección 3.2, donde se presentan el formato y los tipos de mensajes, los intercambios correspondientes a la subida y descarga de archivos y el tratamiento de las condiciones de error.

## **5.4. La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP.**

### **¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características?**

### **¿Cuándo es apropiado utilizar cada uno?**

TCP y UDP proporcionan comunicación entre procesos mediante puertos, pero ofrecen servicios distintos a las aplicaciones.

#### **5.4.1. UDP**

UDP (User Datagram Protocol) es un protocolo orientado a datagramas, con una cabecera de 8 bytes. Sus principales características son:

- No requiere establecer una conexión ni mantiene estado de conexión en el protocolo.
- Conserva los límites de los mensajes enviados por la aplicación.
- Incluye un checksum para detectar errores; en IPv4 su uso es opcional. Si la verificación detecta un error, el datagrama se descarta, sin que UDP solicite su retransmisión.
- No garantiza entrega, orden ni ausencia de duplicados.
- No incorpora control de flujo ni de congestión.

Es apropiado cuando se busca evitar el establecimiento de una conexión, controlar desde la aplicación el envío y la recuperación de datos o priorizar información reciente frente a la retransmisión de información que ya perdió utilidad. Se utiliza, por ejemplo, en consultas DNS y en aplicaciones de voz, vídeo y juegos en tiempo real. También puede servir de base para protocolos confiables, como ocurre en este trabajo. Su menor sobrecarga no implica una garantía de latencia.

#### **5.4.2. TCP**

TCP (Transmission Control Protocol) es un protocolo orientado a conexión que ofrece un flujo bidireccional de bytes. Sus principales servicios y características son:

- Entrega confiable y en orden, mediante números de secuencia, confirmaciones y retransmisiones, evitando entregar datos duplicados a la aplicación.
- Control de flujo, para adecuar el envío a la capacidad de recepción del otro extremo.
- Control de congestión, para adaptar la cantidad de datos en tránsito a las condiciones de la red.
- Establecimiento y cierre de conexión, con mantenimiento de estado en ambos extremos.
- Detección de errores mediante checksum.

A diferencia de UDP, no conserva los límites de los mensajes: corresponde a la aplicación delimitar la información dentro del flujo de bytes. Resulta apropiado para transferencias de archivos, correo electrónico y otros intercambios que requieren recibir los datos completos y ordenados. La confiabilidad no implica que una transferencia pueda completarse ante una falla permanente de la red. TCP tampoco garantiza tiempos de entrega ni proporciona por sí solo cifrado o autenticación.

## **5.5. Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión, en caso de tenerlos, descríbalos.**

El protocolo desarrollado no implementa un mecanismo de control de congestión. Stop & Wait limita el envío a un segmento pendiente de confirmación, mientras que SACK permite mantener hasta 64

segmentos sin confirmar. Estos límites no se ajustan dinámicamente mediante una ventana de congestión en respuesta al estado de la red.

Ambas estrategias estiman el tiempo de espera para retransmisiones a partir del RTT y aumentan ese tiempo cuando se producen vencimientos. Por lo tanto, sí existe una adaptación de los temporizadores que puede reducir la frecuencia de los reintentos, pero no un mecanismo completo de control de congestión como el de TCP. Las confirmaciones selectivas y las retransmisiones permiten recuperar pérdidas; por sí solas no regulan la carga ofrecida a la red en función de su capacidad disponible.

## **5.6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?**

En todas las mediciones realizadas SACK transfirió archivos en menos tiempo en condiciones comparables, especialmente cuando se requiere enviar muchos segmentos. Stop & Wait admite un único segmento sin confirmar, por lo que debe esperar su ACK antes de continuar. Sin pérdidas y despreciando otros costos, el tiempo necesario para enviar y confirmar  $N$  segmentos es aproximadamente  $N \times \text{RTT}$ . En cambio, nuestra estrategia con SACK permite mantener hasta 64 segmentos en vuelo, aprovechando el tiempo durante el cual se esperan las confirmaciones para enviar otros datos.

Esta ventaja suele hacerse más significativa al aumentar la cantidad de segmentos y el RTT, pero no implica que SACK sea siempre más rápido. Para archivos que caben en un solo segmento no existe una ventaja por el uso de una ventana mayor, y los tiempos también dependen de las pérdidas, las retransmisiones y los costos de procesamiento.

## **6. Dificultades encontradas**

### **6.1. Concurrencia del servidor sobre UDP**

La principal dificultad fue diseñar un servidor capaz de atender múltiples clientes de manera concurrente sobre UDP. En TCP, el servidor acepta cada conexión y obtiene un socket independiente que puede delegar a un hilo para atender al cliente correspondiente. En el diseño con UDP y un único socket de recepción, en cambio, los datagramas de todos los clientes llegan al mismo socket. Si varios hilos leyeran directamente de él, un hilo encargado del cliente A podría recibir un datagrama del cliente B, interfiriendo con el procesamiento de ambas transferencias.

Se evaluaron distintas alternativas, entre ellas reproducir una organización similar a la atención de conexiones TCP. Finalmente, se optó por centralizar la recepción en un despachador que actúa como desmultiplexador: identifica la sesión de origen de cada datagrama y lo deriva a su cola correspondiente. De esta manera, cada hilo procesa únicamente los datos de la sesión que atiende. Esta solución exigió definir una lógica de identificación y gestión de sesiones, pero permitió mantener un único punto de recepción y separar la distribución de datagramas del procesamiento de cada transferencia.

### **6.2. Separación de responsabilidades entre capas**

Otra dificultad fue definir abstracciones con responsabilidades claras para las distintas capas de la aplicación. El objetivo era evitar que las operaciones de subida y descarga de archivos quedaran mezcladas con los mecanismos de establecimiento de sesión, confirmación y retransmisión, representados por mensajes como SYN y ACK.

Para lograr esta separación fue necesario delimitar qué información y qué operaciones debía exponer cada capa. La capa de aplicación se ocupa de las solicitudes, el intercambio del contenido de los archivos y la comunicación de errores, mientras que la capa de transferencia confiable resuelve la entrega ordenada y la recuperación ante pérdidas. Esta organización permite utilizar Stop & Wait o SACK sin trasladar sus particularidades a la lógica de subida y descarga.

### **6.3. Diagnóstico de demoras en las pruebas**

Durante las pruebas se observaron tiempos de transferencia que no coincidían con lo esperado: en algunos casos, un archivo de menor tamaño demoraba más que otro de mayor tamaño. Comprender estas diferencias requirió analizar el tráfico y los intercambios entre ambos extremos, ya que el tiempo total por sí solo no permitía identificar la causa.

Este análisis permitió detectar deficiencias en los mecanismos de retransmisión, la gestión de los tiempos de espera y el tratamiento de las pérdidas de paquetes. La dificultad consistió en relacionar las demoras observadas con el estado del protocolo y distinguir el tiempo dedicado al envío de datos del tiempo consumido en esperas y recuperación ante pérdidas.

## **7. Conclusión**

La realización de este trabajo práctico resultó sumamente revelador, nos permitió entender la importancia que tienen los mecanismos de “fallback” para asegurar una comunicación fiable.

Todo proceso de desarrollo de un protocolo debe encararse con la certeza de que todo lo que puede salir mal saldrá mal eventualmente y que debe tenerse una estrategia lista para cada caso.